

Competitive Programming

Section 18 Editorial: DSU And Minimum Spanning Trees

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	DSU reminder	5
3	A. Network Growth	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Algorithm	6
3.5	Correctness proof	7
3.6	Complexity	7
3.7	Full C++ solution	7
3.8	Full Python solution	8
4	B. Minimum Network	10
4.1	Restatement	10
4.2	Observations	10
4.3	Algorithm	10
4.4	Correctness proof	10
4.5	Complexity	11
4.6	Full C++ solution	11
4.7	Full Python solution	12
5	C. Cluster Split	14
5.1	Restatement	14
5.2	Observations	14
5.3	Algorithm	14
5.4	Correctness proof	14
5.5	Complexity	15
5.6	Full C++ solution	15
5.7	Full Python solution	16
6	Common wrong answers	18
7	Stress testing	19
8	What to carry forward	20
9	D. Cable Savings	21
9.1	Restatement	21
9.2	Observation: keep a minimum spanning forest	21
9.3	Walk-through	21
9.4	Algorithm	22
9.5	Correctness proof	22
9.6	Complexity	23

9.7	Implementation notes	23
9.8	Full C++ solution	23
9.9	Full Python solution	24
9.10	Common wrong answers	25
9.11	Tests that matter	26
10	Stress testing	27
11	What to carry forward	28

1 Editorial overview

This section has three DSU applications:

Problem	Main idea
A. Network Growth	maintain components under edge additions
B. Minimum Network	Kruskal MST
C. Cluster Split	stop Kruskal at k components

2 DSU reminder

DSU maintains connected components under additions.

Operations:

```
find(x): representative of x's component  
union(a,b): merge components if representatives differ
```

Invariants:

```
parent[root] == root  
size[root] is valid only for roots  
two vertices are connected iff find(a) == find(b)
```

Use path compression in `find` and union by size in `union`.

3 A. Network Growth

3.1 Restatement

Start with n isolated computers. After each cable is added, output the number of connected components and the size of the largest component.

3.2 Constraints and traps

The graph changes after every edge, so recomputing components with BFS after each update would be too slow.

Traps:

- repeated edges should not change the answer;
- self-loops may appear in the local tests and should not change the answer;
- component count decreases only after a successful merge.

3.3 Observations

DSU tells us whether the new edge joins two different components.

If it does:

```
components -= 1
new_size = merged component size
largest = max(largest, new_size)
```

If not, nothing changes.

3.4 Algorithm

```
initialize DSU with n singleton components
components = n
largest = 1

for each edge u,v:
    if find(u) != find(v):
        merged_size = union(u,v)
        components -= 1
        largest = max(largest, merged_size)
print components, largest
```

3.5 Correctness proof

Lemma 1. DSU components match graph connected components after every processed edge.

Proof. Initially both have n singleton components. When an edge connects two different graph components, graph connectivity merges exactly those two components, and DSU performs the same union. When an edge lies inside one component, neither graph connectivity nor DSU changes. $\square$

Lemma 2. `components` is the number of connected components after every edge.

Proof. By Lemma 1, DSU components are graph components. The counter starts at n and decreases exactly when two DSU components merge. $\square$

Lemma 3. `largest` is the size of the largest connected component after every edge.

Proof. The only way a component size changes is a successful union. The algorithm compares the merged size against the previous maximum. Unchanged edges cannot create a larger component. $\square$

Theorem. Every printed line is correct.

Proof. Follows from Lemmas 2 and 3. $\square$

3.6 Complexity

Time: almost $O(m)$
Memory: $O(n)$

3.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    int unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return sz[ra];
```

```

        if (sz[ra] < sz[rb]) swap(ra, rb);
        parent[rb] = ra;
        sz[ra] += sz[rb];
        return sz[ra];
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    DSU dsu(n);
    int components = n, largest = 1;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        if (dsu.find(u) != dsu.find(v)) {
            largest = max(largest, dsu.unite(u, v));
            components--;
        }
        cout << components << ' ' << largest << '\n';
    }
    return 0;
}

```

3.8 Full Python solution

```

import sys

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)

```

```

        if a == b:
            return self.size[a]
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return self.size[a]

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    dsu = DSU(n)
    components = n
    largest = 1
    out = []
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        if dsu.find(u) != dsu.find(v):
            largest = max(largest, dsu.union(u, v))
            components -= 1
        out.append(f"{components} {largest}")
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

4 B. Minimum Network

4.1 Restatement

Choose a minimum-cost set of edges connecting all vertices. If the graph is disconnected, print IMPOSSIBLE.

4.2 Observations

This is a minimum spanning tree problem. Kruskal sorts edges by weight and takes an edge exactly when it connects two different DSU components.

Negative edge weights are fine. They simply appear early in sorted order and should be taken if they do not create a cycle.

4.3 Algorithm

```
sort edges by weight
total = 0
used = 0
for edge in sorted order:
    if union(u,v):
        total += weight
        used += 1
if used != n-1:
    print IMPOSSIBLE
else:
    print total
```

4.4 Correctness proof

Lemma 1. Kruskal never chooses an edge that creates a cycle.

Proof. DSU rejects an edge exactly when its endpoints are already in the same chosen-edge component. Adding such an edge would create a cycle. $\square$

Lemma 2. Every chosen Kruskal edge is safe for some MST.

Proof. When Kruskal chooses edge e , its endpoints are in different current components. Consider the cut separating one of those components from the rest. Since edges are processed in increasing weight, no lighter crossing edge remains unprocessed and usable. By the cut property, e is safe. $\square$

Theorem. If the algorithm chooses $n-1$ edges, their total cost is the MST cost; otherwise no spanning tree exists.

Proof. By Lemma 1, $n-1$ chosen edges form a tree connecting all vertices. By Lemma 2, each choice can be extended to an MST, so the final tree is minimum. If fewer than $n-1$ edges are chosen, the graph cannot connect all vertices. $\square$

4.5 Complexity

Time: $O(m \log m)$

Memory: $O(n + m)$

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<tuple<long long, int, int>> edges;
```

```

for (int i = 0; i < m; i++) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    edges.push_back({w, u - 1, v - 1});
}
sort(edges.begin(), edges.end());
DSU dsu(n);
long long total = 0;
int used = 0;
for (auto [w, u, v] : edges) {
    if (dsu.unite(u, v)) {
        total += w;
        used++;
    }
}
if (used != n - 1) cout << "IMPOSSIBLE\n";
else cout << total << '\n';
return 0;
}

```

4.7 Full Python solution

```

import sys

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

```

```

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    edges = []
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        w = next(it)
        edges.append((w, u, v))
    edges.sort()
    dsu = DSU(n)
    total = 0
    used = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            total += w
            used += 1
    print("IMPOSSIBLE" if used != n - 1 else total)

if __name__ == "__main__":
    main()

```

5 C. Cluster Split

5.1 Restatement

Choose minimum total edge cost so that the vertices are divided into exactly k connected groups.

5.2 Observations

This is Kruskal stopped early.

Start with n groups. Each successful union reduces the number of groups by one. To end with k groups, we need exactly $n-k$ successful unions.

The same cut-property reasoning applies: at every step, the cheapest edge joining two current groups is safe.

5.3 Algorithm

```
sort edges by weight
components = n
total = 0
for edge in sorted order:
    if components == k:
        break
    if union(u,v):
        total += weight
        components -= 1
if components == k:
    print total
else:
    print IMPOSSIBLE
```

5.4 Correctness proof

Lemma 1. The algorithm maintains a forest with exactly `components` connected groups.

Proof. Initially there are n isolated vertices. A successful union adds an edge between two different groups, so it merges exactly two groups and creates no cycle. Rejected edges change nothing. $\square$

Lemma 2. Each chosen edge is safe for an optimal forest with the target number of groups.

Proof. At the moment an edge is chosen, it is the lightest available edge crossing between two current DSU components. By the same cut-property exchange used in Kruskal, replacing any heavier edge that connects these parts cannot increase the cost. Thus the choice is safe. $\square$

Theorem. If the algorithm reaches k components, the printed cost is the minimum possible cost for exactly k groups; otherwise it is impossible.

Proof. By Lemma 1, reaching k components gives a valid k -group forest. By Lemma 2, the greedy choices are all safe, so the forest is minimum. If all edges are exhausted before reaching k , no remaining edge can merge the remaining components, so no valid solution exists. $\square$

5.5 Complexity

Time: $O(m \log m)$
Memory: $O(n + m)$

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n, m, k;
if (!(cin >> n >> m >> k)) return 0;
vector<tuple<long long, int, int>> edges;
for (int i = 0; i < m; i++) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    edges.push_back({w, u - 1, v - 1});
}
sort(edges.begin(), edges.end());
DSU dsu(n);
int components = n;
long long total = 0;
for (auto [w, u, v] : edges) {
    if (components == k) break;
    if (dsu.unite(u, v)) {
        total += w;
        components--;
    }
}
if (components != k) cout << "IMPOSSIBLE\n";
else cout << total << '\n';
return 0;
}

```

5.7 Full Python solution

```

import sys

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a

```

```

        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    k = next(it)
    edges = []
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        w = next(it)
        edges.append((w, u, v))
    edges.sort()
    dsu = DSU(n)
    components = n
    total = 0
    for w, u, v in edges:
        if components == k:
            break
        if dsu.union(u, v):
            total += w
            components -= 1
    print("IMPOSSIBLE" if components != k else total)

if __name__ == "__main__":
    main()

```

6 Common wrong answers

- Updating component count on repeated or internal edges.
- Using BFS after every edge addition.
- Sorting MST edges in the wrong direction.
- Forgetting that MST costs may be negative.
- Stopping Cluster Split after choosing k edges instead of when k components remain.

7 Stress testing

For small graphs, DSU results can be checked against BFS after each update.

For MST problems, compare Kruskal against brute force over all subsets when $n \leq 8$, or against Prim's algorithm on connected small graphs.

For Cluster Split, brute force subsets can verify the best forest with exactly k components on tiny graphs.

8 What to carry forward

DSU is the standard tool for connectivity under edge additions. Kruskal is the standard first MST algorithm because DSU tells us whether an edge is safe to add.

The next graph sections move from connectivity to weighted shortest paths, where edge weights represent travel cost rather than construction cost.

9 D. Cable Savings

9.1 Restatement

We may deactivate cables in an undirected weighted graph. The remaining network must preserve the original connectivity relation: if two computers were connected by some path before deactivation, they must still have a path afterwards. Different original connected components do not need to become connected.

We want the greatest total cost of deactivated cables. Cable costs are nonnegative, and n, m can both reach 200000, so checking connectivity after each possible removal is far too slow.

9.2 Observation: keep a minimum spanning forest

Let `total` be the sum of all cable costs. Maximizing removed cost is the same as minimizing the cost of the cables we keep:

```
saved cost = total - kept cost
```

Consider one original connected component. The kept cables only need to keep that component connected. A minimum-cost connected subgraph can be reduced to a tree: if it contains a cycle, removing any edge on that cycle keeps it connected and never increases cost because all costs are nonnegative. Thus we need a minimum spanning tree for each original component.

Together, those trees form a **minimum spanning forest**. Kruskal's algorithm already builds exactly that forest if we simply process every edge and never declare a disconnected graph impossible.

9.3 Walk-through

Suppose the graph has two original components:

```
component 1:  1 --4-- 2 --1-- 3
                \      /
                ---3---

component 2:  4 --7-- 5
```

Process edges in increasing cost:

```
2-3, cost 1: keep
1-3, cost 3: keep
1-2, cost 4: skip; it would make a cycle
4-5, cost 7: keep
```

The kept forest costs $1 + 3 + 7 = 11$. All cable costs total 15, so the maximum saving is:

```
15 - 11 = 4
```

An isolated vertex needs no kept cable and contributes neither kept cost nor saved cost.

9.4 Algorithm

```
read all edges and add their weights to total
sort edges by increasing weight
make a DSU with one component per vertex
kept = 0

for every edge (u, v, w) in sorted order:
    if union(u, v) succeeds:
        kept += w

print total - kept
```

Unlike a connected-MST problem, do not check whether exactly $n - 1$ edges were kept. A disconnected input is valid: Kruskal returns one tree for each component.

9.5 Correctness proof

We prove that the algorithm outputs the maximum possible saved cost.

Lemma 1. In every original connected component, there is an optimal set of kept cables that is a spanning tree.

Proof. Any valid kept subgraph is connected within that component. If it contains a cycle, remove an edge from the cycle. Connectivity remains because the other cycle edges give an alternate route. Since cable costs are nonnegative, removing the edge does not increase kept cost. Repeating this removes all cycles and leaves a connected acyclic subgraph, which is a spanning tree. $\square$

Lemma 2. Kruskal's selected edges have minimum possible total cost among all spanning forests that preserve the original components.

Proof. Restrict attention to one original component. Kruskal processes its edges in the same increasing-cost order as ordinary Kruskal on that component, and only joins different current DSU components. By the cut property, every such selected lightest connecting edge is safe for a minimum spanning tree of that component. Therefore its selected edges form a minimum spanning tree there. Applying this independently to every original component proves that the whole selected forest has minimum total cost. $\square$

Theorem. The algorithm prints the maximum total cost that can be saved.

Proof. By Lemma 1, an optimal valid retained network can be chosen as a spanning forest. By Lemma 2, the algorithm's forest has no greater kept cost than any such valid network. The input's total cost is fixed, so subtracting this minimum kept cost from it yields the greatest possible removed cost. Thus the printed value is optimal. $\square$

9.6 Complexity

Sorting dominates the work:

```
Time:     $O(m \log m)$ 
Memory:  $O(n + m)$ 
```

DSU operations are effectively constant amortized time because of path compression and union by size.

9.7 Implementation notes

- Accumulate `total` and `kept` in `long long` in C++; their values can exceed a 32-bit integer.
- Repeated cables are harmless. At most one can be useful for a particular DSU merge; the rest are skipped once their endpoints are connected.
- Zero-cost cables can be selected normally. Keeping or removing an extra zero-cost cycle edge does not change the answer.
- Do not print `IMPOSSIBLE` for a disconnected graph. The requirement is to preserve each original component, not to connect all vertices together.

9.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;

    explicit DSU(int n) : parent(n + 1), size(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        if (parent[x] == x) return x;
        return parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a);
```

```

        b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<tuple<long long, int, int>> edges;
    long long total = 0;
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        edges.emplace_back(w, u, v);
        total += w;
    }

    sort(edges.begin(), edges.end());
    DSU dsu(n);
    long long kept = 0;
    for (auto [w, u, v] : edges) {
        if (dsu.unite(u, v)) {
            kept += w;
        }
    }

    cout << total - kept << '\\n';
}

```

9.9 Full Python solution

```

import sys

class DSU:
    def __init__(self, n: int) -> None:
        self.parent = list(range(n + 1))
        self.size = [1] * (n + 1)

    def find(self, x: int) -> int:

```

```

    while self.parent[x] != x:
        self.parent[x] = self.parent[self.parent[x]]
        x = self.parent[x]
    return x

def union(self, a: int, b: int) -> bool:
    a = self.find(a)
    b = self.find(b)
    if a == b:
        return False
    if self.size[a] < self.size[b]:
        a, b = b, a
    self.parent[b] = a
    self.size[a] += self.size[b]
    return True

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = []
    total = 0
    index = 2
    for _ in range(m):
        u, v, w = data[index:index + 3]
        index += 3
        edges.append((w, u, v))
        total += w

    edges.sort()
    dsu = DSU(n)
    kept = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            kept += w

    print(total - kept)

if __name__ == "__main__":
    main()

```

9.10 Common wrong answers

- Running a connected-MST solution and printing IMPOSSIBLE for an input with multiple original components.
- Summing only skipped edges without accounting for every cable in `total`.
- Using `int` for the sum of up to 200000 costs of 10^9 .

- Sorting in decreasing order, which keeps an expensive forest and minimizes the saving instead.
- Removing a cable whenever its endpoints are currently connected without considering edge costs; the kept forest must be minimum cost.

9.11 Tests that matter

- a graph with no cables and several isolated vertices, which saves 0;
- one tree, where every cable is necessary and saves 0;
- repeated cables with different costs;
- zero-cost edges and cycles;
- multiple disconnected original components;
- a large graph whose total cost requires 64-bit arithmetic.

10 Stress testing

For tiny graphs, enumerate every subset of cables to keep. For each subset, run ordinary BFS from every vertex before and after removals and keep only subsets with exactly the same reachability relation. The best value of

```
total cost - retained cost
```

is an independent oracle for Kruskal's answer. The local generator uses at most ten edges for this exhaustive check and separately creates a 200000-edge case to exercise sorting and DSU at the intended scale.

11 What to carry forward

Kruskal is useful even when the goal is not phrased as “find an MST.” When a problem asks to preserve connectivity while discarding cost, translate it to:

```
minimum cost needed to keep = minimum spanning forest
maximum removable cost      = total cost - forest cost
```

For disconnected inputs, think component by component rather than forcing a single spanning tree.